

Bem-vindos! Já vamos começar

1. Anote o número do seu computador: na quarta, você vai usar o mesmo
2. Na Área de Trabalho, crie uma pasta com o seu nome (exemplo: `minicurso-apis-pedro`)
3. Abra o VS Code > Arquivo > Abrir Pasta > escolha essa pasta
4. Instale a extensão Python: Extensões (Ctrl+Shift+X) > busque "Python" > Microsoft > Instalar
5. Não faça login nas suas contas pessoais nestes computadores

Trouxe notebook? Baixe e rode o `verificar_ambiente.py`:

tinyurl.com/secompp-apis

Consumo de APIs e Análise de Dados na Prática

Felipe W. Giovaneli e Victor B. Zucherato

Quem somos

- **Felipe Giovaneli** — Aluno do 5º ano da computação. Preciso de horas de ACC.
- **Victor Zucherato** — Aluno do 5º ano da computação. Desenvolve o TCC com dados coletados da API do Brawl Stars

Até quarta vocês vão
escrever isto

—





#RGVU2J



Vovoito





Vovoito





Vovoito
Royalty





TROPHY ROAD

 **111152**

Highest: 111159



BRAWLERS

106 / 106 Collected

105
TOTAL
PRESTIGE



SHOW MORE

MENU

RANKED





#RGVU2J

Vovoito

Vovoito

105

Vovoito
Royalty

37 MAX
WIN STREAK

15310



SHOW MORE

WINS

SHOWDOWN

5579

3v3 WINS

21001

RANKED



CURRENT
2349



HIGHEST
8176



PHANTOM LORD'
Member





Most Trophies



SEARCH



BRAWLERS 106/107



O plano

Segunda (28/09)	Quarta (30/09)
O que é uma API e como a web conversa	Revisão rápida da segunda
Primeira requisição em Python	Análise de dados com pandas
JSON e dicionários	Desafio livre: vocês escolhem a API
Brawl Stars com chave de acesso	Apresentação dos resultados

Combinados

- Travou? Levante a mão: um de nós vai até você
- Erro não é fracasso. Ler o erro é metade do trabalho de quem programa
- Todo o código está no repositório: **tinyurl.com/secompp-apis**. Se se perder, copie de lá e siga a turma
- Mesmo computador nos dois dias

Fundamentos de APIs, HTTP e Comunicação Web

API



Application Programming Interface

O que é API?

Imagine um
restaurante

- Cliente
- Cozinha
- Garçom



Relação Cliente e Servidor



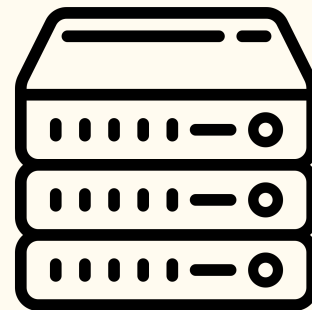
Cliente



Requisição (Request)



Resposta (Response)



Servidor

Relação Cliente e Servidor

Client: Quem solicita as informações (ex: nosso script em Python, navegador, app de celular).

Server: O computador remoto de alta capacidade que processa e armazena os dados.

Request: A mensagem de solicitação enviada do cliente para o servidor.

Response: A mensagem enviada do servidor de volta ao cliente com os dados ou status.

Anatomia de uma URL e Verbos HTTP



Desmembrando uma URL

`https://api.brawlstars.com/v1/rankings/br/players?limit=5`

The diagram illustrates the structure of the URL `https://api.brawlstars.com/v1/rankings/br/players?limit=5` by using brackets to group its parts into four categories: Protocolo, Domínio, Endpoint, and Parâmetros. The labels are positioned below the corresponding segments of the URL.

Protocolo	Domínio	Endpoint	Parâmetros
<code>https://</code>	<code>api.brawlstars.com</code>	<code>/v1/rankings/br/players</code>	<code>?limit=5</code>

Desmembrando uma URL

Protocolo (https://): Define as regras de segurança da comunicação.

Domínio (api.brawlstars.com): O endereço do servidor na internet.

Endpoint (/v1/rankings/br/players): O caminho específico que indica qual recurso ou dado estamos buscando (no caso, o ranking dos jogadores do Brasil).

Parâmetros de Query (?limit=5): Filtros opcionais adicionados após a interrogação (?), separados pelo caractere &, que personalizam o resultado.

Métodos de Comunicação

Verbos HTTP

Verbos HTTP

GET: Lê ou recupera dados de um recurso (Não altera o estado do servidor)

POST: Cria um novo recurso no servidor.

PUT: Substitui ou atualiza um recurso inteiro existente.

PATCH: Atualiza parcialmente um recurso.

DELETE: Apaga um recurso específico.

Outros Verbos HTTP

HEAD: Pede apenas os cabeçalhos de uma resposta, sem o corpo.

OPTIONS: Mostra quais métodos de comunicação são aceitos pelo recurso.

CONNECT: Cria um túnel de rede para o servidor.

TRACE: Faz um teste de eco da requisição recebida.

Status Codes

A Primeira Linha de Diagnóstico

| Status Codes

200	OK	Sucesso! O servidor entendeu o pedido e enviou os dados esperados.
400	Bad Request	Erro do cliente. O formato da URL ou os parâmetros enviados contêm erros de digitação.
401	Unauthorized	Falta de identificação. Você não enviou o Token/Chave da API ou ele está ausente.
403	Forbidden	Acesso negado. Sua chave existe, mas você não tem permissão para acessar aquele recurso (ex: IP bloqueado).
404	Not Found	Não encontrado. O endpoint digitado não existe ou a TAG do jogador/item busca não foi localizada.
429	Too Many Requests	Você estourou o Rate Limit. Enviou requisições demais em um curto espaço de tempo.
500	Internal Server Error	Erro interno do servidor. A culpa não é sua; a API deles está instável ou fora do ar.

Headers, Autenticação e Rate Limit

Headers e Token de Autenticação

Headers (Cabeçalhos): São informações invisíveis enviadas junto com a requisição, funcionando como uma etiqueta de envio. Informam o tipo de dado que aceitamos receber (Accept: application/json) ou identificam quem está pedindo.

Bearer Token / API Key: O seu "crachá de acesso". APIs comerciais ou restritas exigem que você envie essa chave secreta dentro do Header (Authorization: Bearer SUA_CHAVE) para contabilizar quem está consumindo seus dados.

Rate Limit

As APIs impõem um limite máximo de requisições por segundo ou por minuto para evitar que usuários mal-intencionados ou scripts sem controle travem os servidores com excesso de tráfego.

Caso você ultrapasse esse limite, o servidor retorna o código 429 e bloqueia novas chamadas temporariamente.

Mãos à obra

https://viacep.com.br/ws/SEU_CEP/json/
8 números, sem hífen (exemplo: 19060900)

<https://pokeapi.co/api/v2/pokemon/NOME/>
em minúsculas (exemplo: pikachu)

Prática 1

A primeira requisição em Python

Do navegador para o Python

Crie o arquivo **01_primeira_requisicao.py** dentro da sua pasta

```
import requests

url = "https://viacep.com.br/ws/19060900/json/"
resposta = requests.get(url)

print(resposta.status_code)
print(resposta.json())
```

Sempre confira o status primeiro

Apague os dois `print` do final e coloque no lugar:

```
if resposta.status_code == 200:
    dados = resposta.json()
    print("Rua:", dados["logradouro"])
    print("Bairro:", dados["bairro"])
    print("Cidade:", dados["localidade"])
else:
    print("Deu erro! Status:", resposta.status_code)
```

Nem todo 200 é sucesso

Teste estes dois CEPs e compare:

- **12345** → o que acontece com o status?
- **99999999** → o status é 200... mas olha o que veio dentro

Nem todo 200 é sucesso

Teste estes dois CEPs e compare:

- **12345** → o que acontece com o status?
- **99999999** → o status é 200... mas olha o que veio dentro

Resposta:

- 12345 tem formato inválido: a API devolve **400**
- 99999999 tem formato válido, mas não existe: a API devolve **200** com `{"erro": "true"}`

Conferindo o conteúdo também

Troque o `if/else` por este:

```
if resposta.status_code == 200:
    dados = resposta.json()
    if "erro" in dados:
        print("CEP não encontrado!")
    else:
        print("Rua:", dados["logradouro"])
        print("Bairro:", dados["bairro"])
        print("Cidade:", dados["localidade"])
else:
    print("Deu erro! Status:", resposta.status_code)
```

Terminou? Desafios

1. Busque o CEP da sua casa
2. Na PokeAPI, puxe um pokémon e mostre o nome, a altura e o peso
3. Descubra a unidade: o peso do pikachu vem como 60. São 60 kg?

Formato JSON, Dicionários e Navegação de Dados em Python

Felipe Giovaneli e Victor Zucherato

JSON vs. Dicionário Python

JSON



JavaScript Object Notation

Dicionário Python

É a estrutura de dados nativa em memória (chave-valor) que nos permite manipular a informação.

A Ponte

`resposta.json()`: O método `.json()` da biblioteca `requests` lê o texto bruto enviado pelo servidor e o converte automaticamente em um Dicionário (ou Lista) nativo do Python.

Texto JSON na Web —> [`resposta.json()`] —> Dicionário Python
'{"nome": "Bull"}' (A Ponte) {"nome": "Bull"}

Mapeamento de Tipos e Aninhamento

Estruturas complexas possuem dicionários dentro de listas,
que estão dentro de outros dicionários!

| Equivalência

Estrutura no JSON	Tipo em Python	Exemplo
Objeto {}	Dicionário (dict)	{"trofeus": 1200}
Lista []	Lista (list)	["Shelly", "Colt", "Nita"]
Texto "..."	String (str)	"Brawl Stars"
Número	Inteiro (int) / Decimal (float)	1200 / 3.14
Booleano true / false	Booleano (True / False)	True
Nulo null	Nulo (None)	None

Acessando Dados

Abordagem tradicional
(colchetes[])

```
jogador = {"nome": "Spike", "nivel": 10}

# Acesso por colchetes
print(jogador["nome"])    # Saída: Spike
print(jogador["clube"])    # KeyError: 'clube'
```



Acessando Dados

Acesso Seguro com
.get()

```
# Se a chave não existir, retorna None em vez de quebrar:  
clube = jogador.get("clube") # Saída: None  
  
# Definindo um valor padrão caso a chave não exista:  
clube = jogador.get("clube", "Sem clube") # Saída: Sem clube
```

Navegando em dados aninhados

```
jogador = {  
    "name": "Victor",  
    "club": {"name": "Clube da FCT"},  
    "brawlers": [  
        {"name": "SHELLY", "trophies": 750},  
        {"name": "COLT", "trophies": 620},  
    ],  
}  
  
print(jogador["club"]["name"])      # Clube da FCT  
print(jogador["brawlers"][0]["name"]) # SHELLY  
  
for brawler in jogador["brawlers"]:  
    print(brawler["name"], brawler["trophies"])
```

Como Ler JSONs Grandes Sem Enlouquecer

Formatando a saída no Python

```
import json

# Transforma o dicionário em texto formatado com recuo
print(json.dumps(dados_api, indent=2, ensure_ascii=False))
```

Soluções no Navegador

Extensões do Navegador: JSON Viewer / JSON Formatter.

Ferramentas Online: Colar o retorno em manipuladores visuais para abrir/fechar blocos de dados.

Mãos à obra

Prática 2



Entrando no Brawl Stars

A API do Brawl Stars

- Oficial, gratuita e mantida pela Supercell: **developer.brawlstars.com**
- Exige uma chave de acesso (o "crachá" da Teoria 1)
- Cada chave só funciona a partir dos IPs cadastrados nela
- Hoje vocês vão usar a chave do minicurso, criada para o IP deste laboratório. Ela será apagada na quarta

Pegue a chave

- Acesse: **link retirado desta versão**
- O arquivo tem uma linha só: `TOKEN = "..."`. Copie tudo com Ctrl+A e Ctrl+C: é um texto enorme, não tente digitar
- No próximo código, cole essa linha no lugar de `TOKEN = "COLE_A_CHAVE_AQUI"`

Mandando o crachá junto com o pedido

Crie o arquivo **02_brawlstars.py**

```
import requests

TOKEN = "COLE_A_CHAVE_AQUI"
cabecalhos = {"Authorization": f"Bearer {TOKEN}"}

url = "https://api.brawlstars.com/v1/rankings/br/players?limit=5"
resposta = requests.get(url, headers=cabecalhos)

print(resposta.status_code)
print(resposta.json())
```

O problema da

- No jogo, a tag é **#RGVU2J**
- Na URL, ela vira **%23RGVU2J**
- Motivo: numa URL, tudo que vem depois de # nunca chega ao servidor. %23 é o jeito codificado de escrever #

```
tag = "#RGVU2J"  
tag_url = tag.replace("#", "%23")  
url = f"https://api.brawlstars.com/v1/players/{tag_url}"
```

Buscando um jogador

```
resposta = requests.get(url, headers=cabecalhos)

if resposta.status_code == 200:
    jogador = resposta.json()
    print(jogador["name"])
    print(jogador["trophies"])
else:
    print("Erro:", resposta.status_code)
    print(resposta.json())
```

Navegando no JSON do jogador

```
print("Nome:", jogador["name"])
print("Troféus:", jogador["trophies"])
print("Recorde:", jogador["highestTrophies"])
print("Clube:", jogador.get("club", {}).get("name", "Sem clube"))
print("Brawlers:", len(jogador["brawlers"]))

for brawler in jogador["brawlers"]:
    print(brawler["name"], "-", brawler["trophies"], "troféus")
```

Guardando os dados para quarta

```
import json

with open("jogador.json", "w", encoding="utf-8") as arquivo:
    json.dump(jogador, arquivo, ensure_ascii=False, indent=2)

print("Arquivo salvo!")
```

Terminou? Desafios

1. Troque a tag pela sua (ou de um amigo que joga)
2. Qual é o seu brawler com mais troféus?
3. Puxe o top 10 do Brasil e mostre só o nome e os troféus de cada um
4. **Difícil:** no endpoint `/battlelog`, conte quantas vitórias e derrotas o jogador teve nas últimas partidas

Hoje vocês aprenderam a

- Entender o que acontece entre o pedido e a resposta
- Fazer requisições em Python e ler o status
- Navegar em JSON como dicionário
- Se autenticar numa API com chave

Quarta (30/09), às 8h

- Sente no **mesmo computador** e não apague a sua pasta
- Vamos transformar o **jogador.json** em tabela com o pandas
- Já pensem em qual API vocês querem usar no desafio livre: PokeAPI, Rick and Morty, Brasil API, Harry Potter API, Open-Meteo...
- Código e dados de exemplo: **tinyurl.com/secomp-api**

Para quem quiser continuar em casa

- A chave do minicurso não funciona fora daqui. Crie a sua em `developer.brawlstars.com`, com o IP da sua casa
- Sem Brawl Stars: escolha 5 pokémons e descubra qual é o mais pesado

Bônus



Se sobrar tempo

Status code na marra

Antes de rodar, apostem: que status cada URL vai devolver?

```
import requests

urls = [
    "https://viacep.com.br/ws/19060900/json/",
    "https://viacep.com.br/ws/12345/json/",
    "https://pokeapi.co/api/v2/pokemon/naoexiste",
    "https://api.brawlstars.com/v1/players/%23RGVU2J",
]

for url in urls:
    resposta = requests.get(url)
    print(resposta.status_code, url)
```

Parâmetros sem montar a URL na mão

Os filtros vão num dicionário e o `requests` monta a URL:

```
import requests

parametros = {
    "latitude": -22.12,
    "longitude": -51.39,
    "daily": "temperature_2m_max",
    "timezone": "America/Sao_Paulo",
}

url = "https://api.open-meteo.com/v1/forecast"
resposta = requests.get(url, params=parametros)

print(resposta.url)  # o ? e os & foram montados sozinhos
print(resposta.json()["daily"]["temperature_2m_max"])
```

Os mesmos parâmetros no Brawl Stars

A chave continua indo nos headers:

```
url = "https://api.brawlstars.com/v1/rankings/br/players"
parametros = {"limit": 10}
resposta = requests.get(url, headers=cabecalhos, params=parametros)

for jogador in resposta.json()["items"]:
    print(jogador["rank"], jogador["name"], jogador["trophies"])
```

E se a internet cair?

Com `try` e `except`, o erro vira uma mensagem clara:

```
import requests

try:
    url = "https://viacep.com.br/ws/19060900/json/"
    resposta = requests.get(url, timeout=5)
    resposta.raise_for_status()
    print("Cidade:", resposta.json()["localidade"])
except requests.exceptions.Timeout:
    print("A API demorou demais para responder")
except requests.exceptions.ConnectionError:
    print("Sem conexão: confira a internet")
except requests.exceptions.HTTPError as erro:
    print("A API respondeu com erro:", erro)
```

Provoque cada erro

1. Troque o `timeout=5` por `timeout=0.001`: cai no primeiro `except`
2. Troque `viacep.com.br` por `viacep.com.brr`: cai no segundo
3. Troque o CEP por `12345`: cai no terceiro, com o 400 que vocês já conhecem

Antes de sair

Este computador é usado por todos os minicursos da semana.

- Saia de qualquer conta em que você entrou
- Feche o navegador